

r_chatbot — how it works and how it connects to langgraphr

Note

This report was generated using AI under general human direction. At the time of generation, the contents have not been comprehensively reviewed by a human analyst.

What the chatbot is

`r_chatbot` is a small conversational assistant written **entirely in R**. You type messages in a console, or in a Shiny chat window, and the bot — “Ada” — answers with full memory of the conversation.

The interesting part is what happens under the hood: the bot is not an R implementation of an AI agent. It delegates the AI work to the **langgraphr** package, which runs a graph engine in a hidden local background server. Your R code and your conversation never leave your machine — the only thing that travels to the AI provider is the message text sent by the hidden server when it needs a model reply.

The chatbot can:

- Chat with an OpenAI-compatible model (configured in `.env`)
- Remember the whole conversation (in-process, or durably in SQLite)
- Call **one R tool** (`session_info_tool`) when the user asks about their R environment
- Compress its own history every 10 messages so the context stays short

It cannot (by design): touch your data, files, or databases. It is chat + memory + one info tool.

How it connects to langgraphr

The chatbot uses langgraphr’s **assistant path**: a ready-made conversational agent created with `lg_connect()`.

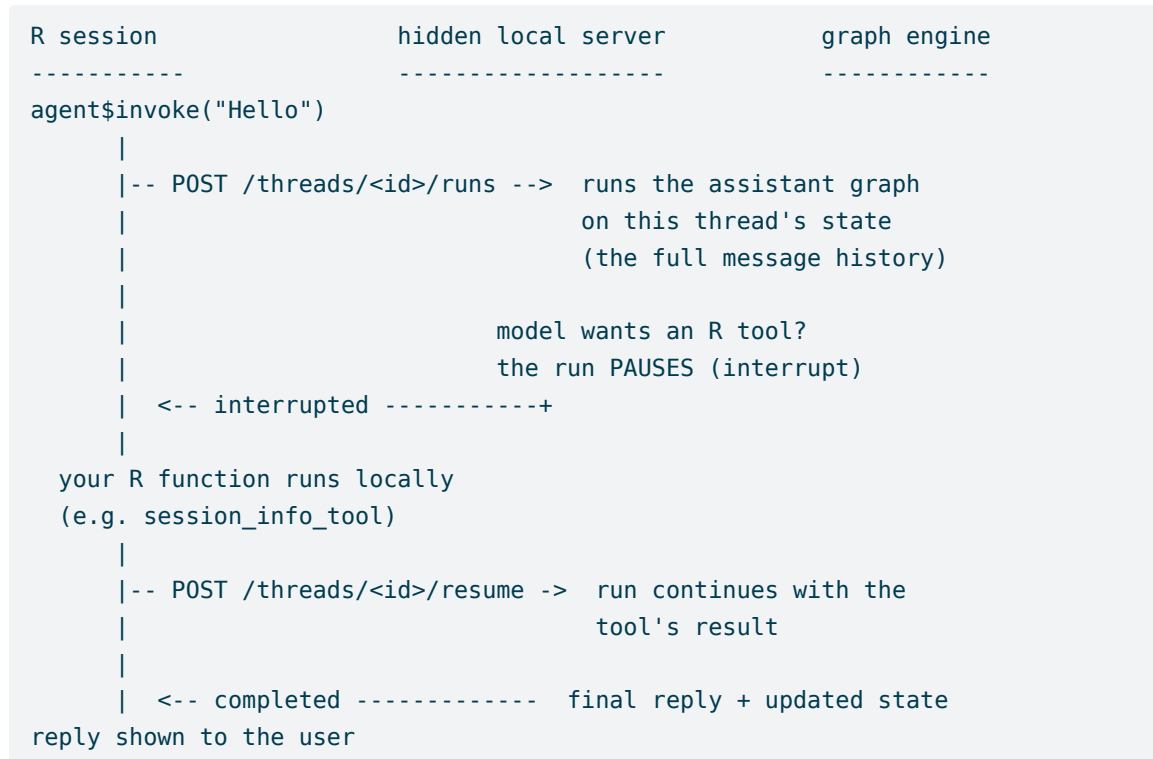
```
agent <- lg_connect()           # starts (or reuses) the hidden server
agent$add_tool(session_info_tool,
                description = "Report the user's R environment...")
res <- agent$invoke("Hello!")  # one full conversational turn
```

`lg_connect()` does two things. First, it makes sure the hidden server is running — if not, it starts it automatically as a supervised background process. Second, it returns an `LgAgent` object bound to a `thread_id`, which is the memory key for the conversation.

From that moment, the chatbot never manages any infrastructure. It only talks to langgraphr through three calls: `add_tool()`, `invoke()`, and `reset()`.

The round trip of one message

When you type “Hello” and press Enter, this is the complete journey:



Two properties of this design matter:

1. **R always starts the HTTP call.** The hidden server never calls back into R on its own, so there is no deadlock and no listener to set up.
2. **R tools run in your session.** When the model asks for a tool, the run pauses over HTTP, the R function executes locally with your environment, and the result is sent back. The engine only decides *when* to call a tool — the tool itself is ordinary R code.

How a turn is produced

The Shiny front end runs each turn in a background R process so the interface never freezes while the model is thinking. The whole turn is one self-contained function in `R/graph.R`:

```
chat_turn <- function(msg, thread_id, turn_count, personality,
                      seed = NULL, tools = list(), summary_every = 10L,
                      env_path = NULL, durable = FALSE, db_path = NULL) {
  library(langgraphr)
  # load credentials, enable SQLite memory when asked
```

```

if (!is.null(env_path) && file.exists(env_path)) {
  try(dotenv::load_dot_env(env_path), silent = TRUE)
}
if (isTRUE(durable)) {
  if (is.null(db_path)) db_path <- "chatbot_memory.sqlite"
  lg_use_sqlite(db_path)
}
# reconnect to the SAME thread: the server restores this conversation
agent <- lg_connect(thread_id = thread_id)
for (t in tools) agent$add_tool(t$fn, description = t$description)
# fresh thread: seed the personality first
if (!is.null(seed) && nzchar(seed)) invisible(agent$invoke(seed))
# the user's actual message
res <- agent$invoke(msg)
# summarize step (see below)
...
list(content = res$content, note = note, thread_id = agent$thread_id)
}

```

The personality is not a special API feature — it is seeded as the **first message** of every fresh thread (“System note: you are Ada...”), so the instruction lives in the remembered history like every other message.

How memory works

Memory is keyed by `thread_id`. Sending every message with the same id continues the same conversation, because the hidden server keeps each thread’s full state — the message history — for as long as the server runs.

Two extras sit on top of that:

Durable memory. Calling `lg_use_sqlite("chatbot_memory.sqlite")` before the server starts points the engine at a SQLite file. Every thread’s state is then written to disk, so conversations survive server restarts and R restarts. The Shiny app also saves the current thread id to `chatbot_thread_id.txt`, so re-opening the app resumes the same conversation; “New conversation” writes a fresh id and clears the view.

The summarize step. After every 10 user messages, the bot compresses its own history: it asks the model to summarize the current thread, switches to a new thread with `reset()`, and carries the summary in as the first message. Key facts survive; the context stays short.

The two front ends

The brain lives in `R/graph.R`, `R/tools.R` and `R/utils.R`; two thin front ends share it.

Front end	File	How it talks to langgraphr
Console	<code>main.R</code>	a <code>readline()</code> loop; <code>agent\$invoke()</code> runs in the same process; <code>quit/exit</code> leave, <code>clear</code> resets memory
Shiny window	<code>app.R</code>	<code>shinychat</code> bubbles; each <code>invoke()</code> runs in a background R process via <code>callr</code> + promises, so the UI stays responsive while the model thinks

The Shiny background design matters: a model reply takes seconds, and a blocking call would freeze the whole Shiny session long enough for the browser to show a “connection lost” overlay. Running the turn in a worker process keeps the interface live.

Running it

One-time setup:

```
remotes::install_local("LanggraphR") # install the package from source
install.packages(c("dotenv", "R6", "cli", "shinychat"))
```

Put your credentials in `projects/r_chatbot/.env`:

```
LANGGRAPH_MODEL=deepseek-v4-flash
LANGGRAPH_API_KEY=sk-...
LANGGRAPH_BASE_URL=https://api.deepseek.com
```

Then, in an R session:

```
# console chat
source("projects/r_chatbot/main.R")

# or the chat window
shiny::runApp("projects/r_chatbot/app.R")
```

The first connection takes a few extra seconds (the hidden server starts and the personality is seeded). After that, each reply is a normal model round-trip.